Deadlock

1. Deadlock Overview

1.1 Why Learn This?

Deadlock is the **most hidden and most fatal** concurrency bug in multi-tasking programming. It has three terrifying aspects:

1. **Cannot reproduce** — Deadlock depends on precise task timing; testing 1000 times may only trigger once. Your development board runs for three days without deadlock, but the customer site deadlocks after 30 minutes.
2. **Cannot recover** — Once deadlocked, related tasks are permanently stuck, **no timeout, no watchdog reset, no recovery mechanism can automatically resolve**. The only fix is to reboot.
3. **Cannot locate** — System doesn't crash, doesn't print errors, doesn't trigger exceptions during deadlock. Product manifestation is "some function suddenly unresponsive," you can't even find an entry point for debugging.

1.2 Real-World Painful Lessons

Event	Year	Consequence
NASA Mars Pathfinder	1997	Priority inversion caused frequent system resets, nearly mission failure after landing. NASA engineers remotely patched to save it
Toyota Prius brake gate	2010	Multi-task lock competition caused brake response delay, global recall over 100,000 vehicles
Some domestic drone crash	2020	Flight control task deadlock, attitude correction failed within 0.8 seconds, direct crash

> Deadlock doesn't boldly appear during development. It hides behind thousands of normal runs, waiting until your product is shipped and customers complain, then starts to manifest.

1.3 What This Project Demonstrates

This project reproduces the most classic and easiest-to-make deadlock scenario: **Task1 takes lock1 then lock2, Task2 takes lock2 then lock1** — only the acquisition order is reversed, everything else is identical. Just this "reversed order" difference causes two tasks to form circular wait, each stuck at the second `xSemaphoreTake`, **the code after will never execute**.

---

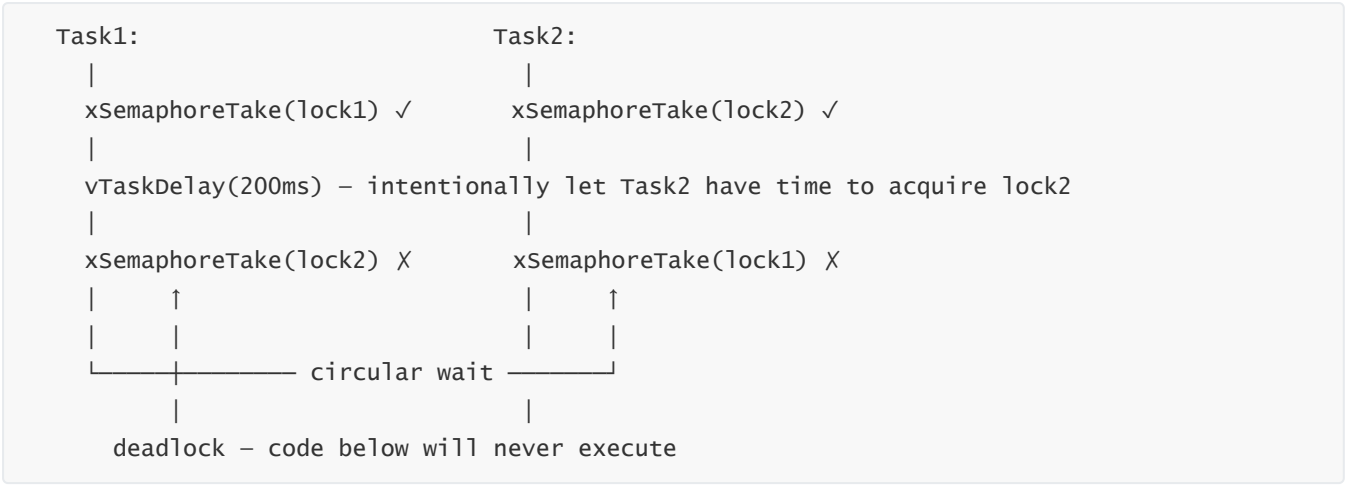
2. Core Concepts

2.1 Four Necessary Conditions for Deadlock

Condition	Meaning	How This Project Satisfies
Mutual exclusion	Resource can only be held by one task	Mutex can only be Take by one task at a time
Hold and wait	Holds one resource while waiting for another	Task1 holds lock1, waits for lock2
No preemption	Resource cannot be forcibly released	Mutex cannot be forcibly given by other tasks
Circular wait	Tasks form closed loop waiting for each other	Task1 waits for Task2's lock2, Task2 waits for Task1's lock1

All four conditions satisfied = deadlock.

2.2 Deadlock Diagram



2.3 Mutex API

The following are mutex creation, acquire, and release APIs:

```c
// Create mutex
SemaphoreHandle_t xSemaphoreCreateMutex(void);

// Take lock (block forever)
xSemaphoreTake(lock, portMAX_DELAY);

// Give lock
xSemaphoreGive(lock);
```

---

3. Project Architecture and Flow

```
app_main()
  ├─ xSemaphoreCreateMutex() → lock1
  ├─ xSemaphoreCreateMutex() → lock2
  ├─ xTaskCreate(task1) → Core 0, Prio 1
  └─ xTaskCreate(task2) → Core 0, Prio 1

  Both tasks pinned to Core 0 – same core serial scheduling, deadlock inevitable

  task1: take lock1 → Delay 200ms → try lock2 (blocks forever)
  task2: take lock2 → Delay 200ms → try lock1 (blocks forever)
```

Key Design: The two 200ms delays are intentionally left as time windows to ensure that Task1 and Task2 each acquire their first lock before simultaneously requesting the other's lock.

---

4. Hardware Dependencies

This project is a **pure software demonstration** with no hardware peripheral dependencies (only serial log output).

Required Components: `freertos/FreeRTOS`, `freertos/task`, `freertos/semphr`, `esp_log`

---

5. Source Code Analysis

Full source code: `Source Code → ESP-IDF → 4.FreeRTOS → 6. RTOS_Deadlock`

5.1 Header Files and Macro Configuration

This project includes FreeRTOS task, semaphore related headers and logging module, and declares two global mutexes for deadlock demonstration.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_log.h"

static const char *TAG = "DEADLOCK";    // Log tag

// Two global mutexes (used to create deadlock)
SemaphoreHandle_t lock1;
SemaphoreHandle_t lock2;
```

5.2 task1() — Take lock1 then lock2

task1 first acquires lock1, after 200ms delay tries to acquire lock2. By this time lock2 is already held by task2, task1 blocks permanently at the second lock acquisition, forming deadlock.

```
void task1(void *pv)
{
    while (1) {
        ESP_LOGI(TAG, "Task1 waiting for lock1");

        // Block forever until lock1 is acquired
        xSemaphoreTake(lock1, portMAX_DELAY);
        ESP_LOGI(TAG, "Task1 acquired lock1");

        // Delay 200ms, intentionally let task2 acquire lock2
        vTaskDelay(pdMS_TO_TICKS(200));

        ESP_LOGI(TAG, "Task1 waiting for lock2...(Deadlock starts)");

        // Will never acquire lock2 (held by task2) → deadlock
        xSemaphoreTake(lock2, portMAX_DELAY);

        // ★ The code below will never execute ★
        ESP_LOGI(TAG, "========= This line will never be printed =========\n");

        xSemaphoreGive(lock2);
        xSemaphoreGive(lock1);
    }
```

```
}
```

5.3 task2() — Take lock2 then lock1

task2's acquisition order is **opposite** to task1 — takes lock2 first, then lock1. This is the direct cause of deadlock.

```
void task2(void *pv)
{
    while (1) {
        ESP_LOGI(TAG, "Task2 waiting for lock2");

        // Block forever until lock2 is acquired
        xSemaphoreTake(lock2, portMAX_DELAY);
        ESP_LOGI(TAG, "Task2 acquired lock2");

        // Delay 200ms, intentionally let task1 acquire lock1
        vTaskDelay(pdMS_TO_TICKS(200));

        ESP_LOGI(TAG, "Task2 waiting for lock1...(Deadlock starts)");

        // Will never acquire lock1 (held by task1) → deadlock
        xSemaphoreTake(lock1, portMAX_DELAY);

        // ★ The code below will never execute ★
        ESP_LOGI(TAG, "========= This line will never be printed =========\n");

        xSemaphoreGive(lock1);
        xSemaphoreGive(lock2);
    }
}
```

5.4 app_main() — Main Entry

app_main creates two mutexes, then pins both tasks to the same core, ensuring deadlock inevitably triggers under single-core serial scheduling.

```
void app_main(void)
{
    // Create two mutexes
    lock1 = xSemaphoreCreateMutex();
    lock2 = xSemaphoreCreateMutex();

    // ★ Both tasks pinned to same core → deadlock inevitable ★
    xTaskCreatePinnedToCore(task1, "task1", 2048, NULL, 1, NULL, 0);
    xTaskCreatePinnedToCore(task2, "task2", 2048, NULL, 1, NULL, 0);
}
```

> **Why pin to same core?** Ensure single-core serial scheduling, deadlock inevitably occurs. Cross-core scenarios may occasionally avoid deadlock due to timing differences (but still unsafe).

---

6. Operation Flow

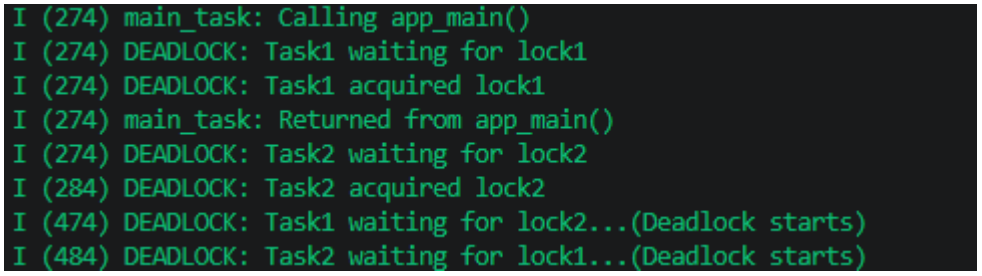
6.1 Compilation and Flashing Flow

For detailed flashing process, see: 1. Before Development → 3. Build and Flash

6.2 Observe Deadlock

After flashing, power on, observe serial logs:

```
I (1263) DEADLOCK: Task1 waiting for lock1
I (1263) DEADLOCK: Task1 acquired lock1              ← Task1 gets lock1
I (1263) DEADLOCK: Task2 waiting for lock2
I (1263) DEADLOCK: Task2 acquired lock2              ← Task2 gets lock2
I (1463) DEADLOCK: Task1 waiting for lock2...(Deadlock starts)   ← Task1 tries lock2
I (1463) DEADLOCK: Task2 waiting for lock1...(Deadlock starts)   ← Task2 tries lock1
                                                      ← ★ No more output after this ★
```



6.3 Key Characteristics of Deadlock

What you observe after deadlock occurs:

- **Logs stop at last two lines** — No ERROR, WARNING, assert, or panic. System doesn't crash, doesn't error-report, doesn't restart.
- **Two tasks disappeared** — They're still in memory, but not scheduled. You can't type commands to check status (no shell), can't set breakpoints (CPU isn't running your code).
- **Other system parts look "normal"** — app_main's `while(!g_game_over)` still runs (it doesn't read those locks), but task1/task2 are permanently disabled.

> In real products, this "silent disability" is the most fatal — your device is still running, serial port is still printing heartbeat packets, but some critical function is no longer responding. By the time you notice, the device may have been running for three days and three nights on site, with no clues in the logs.

6.4 Comparison: If Lock Acquisition Order Was Consistent

If task2 also follows lock1 → lock2 order (consistent with task1):

```
Task1: take lock1 ✓ → take lock2 ✓ → execute → release lock2 → release lock1
Task2: take lock1 (wait Task1 release) → take lock1 ✓ → take lock2 ✓ → execute
```

Just change one line of acquisition order, deadlock never happens.

---

7. How to Prevent Deadlock

7.1 Unified Lock Acquisition Order

Most effective deadlock prevention. All tasks acquire locks in the same order.

```
// Correct – unified order: lock1 → lock2
void task_safe(void *pv) {
    xSemaphoreTake(lock1, portMAX_DELAY);  // Acquire lock1 first
    xSemaphoreTake(lock2, portMAX_DELAY);  // Then acquire lock2
    // ... critical section ...
    xSemaphoreGive(lock2);                 // Release lock2 first
    xSemaphoreGive(lock1);                 // Then release lock1
}
```

7.2 Use Timeout

Don't wait for lock indefinitely. After timeout, actively release already-acquired locks to avoid permanent blocking.

```
// Timeout mechanism to prevent deadlock
if (xSemaphoreTake(lock1, pdMS_TO_TICKS(1000)) == pdPASS) {
    if (xSemaphoreTake(lock2, pdMS_TO_TICKS(1000)) == pdPASS) {
        // Both acquired, execute critical section
    } else {
        xSemaphoreGive(lock1);
    }
}
```

7.3 Reduce Lock Hold Time

Keep critical sections as short as possible. **Don't call `vTaskDelay` while holding locks** — this is the most common deadlock trigger.

```
// Wrong – delay while holding lock
xSemaphoreTake(lock, portMAX_DELAY);
// ❌ Lock held idle for 200ms
vTaskDelay(pdMS_TO_TICKS(200));
g_counter++;
xSemaphoreGive(lock);

// Correct – only protect necessary shared resource access
xSemaphoreTake(lock, portMAX_DELAY);
// ✓ Critical section is very short
g_counter++;
xSemaphoreGive(lock);
vTaskDelay(pdMS_TO_TICKS(200));  // ✓ Delay outside the lock
```

7.4 Use Recursive Mutex

The same task can acquire the same lock multiple times (with usage count), avoiding self-deadlock.

```
// Recursive mutex
SemaphoreHandle_t xRecursiveMutex = xSemaphoreCreateRecursiveMutex();
// First acquisition
xSemaphoreTakeRecursive(xRecursiveMutex, portMAX_DELAY);
// Second (won't block)
xSemaphoreTakeRecursive(xRecursiveMutex, portMAX_DELAY);
// Release once
xSemaphoreGiveRecursive(xRecursiveMutex);
// Release twice
xSemaphoreGiveRecursive(xRecursiveMutex);
```

---

8. FAQ

Symptom	Cause	Solution
Logs stop with no further output	**Deadlock occurred** — two tasks permanently blocked	Unified lock acquisition order
Deadlock didn't occur (cross-core)	Cross-core parallel execution timing coincidence	Pin to same core to ensure inevitable trigger
Deadlock after `vTaskDelay`	Switched tasks while holding lock	Don't call any switching APIs while holding locks
Deadlock with only one lock	Recursively acquiring mutex	Use `xSemaphoreCreateRecursiveMutex`